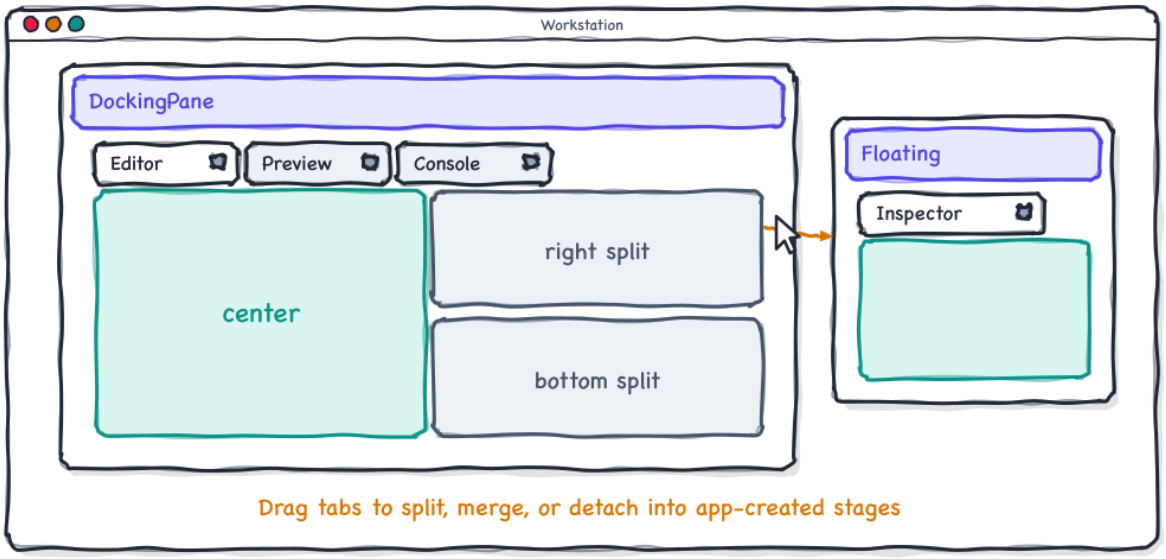
DockingPane

Developer Manual

GemsFX • com.dlsc.gemsfx.docking • JavaFX Custom Controls



Dockables can be grouped as tabs, split into regions, and moved into floating stages.

DockingPane provides IDE-style docking with draggable tabs, split layouts, optional floating windows, and complete context snapshots through DockingContext. This manual covers structure, interaction, stage hosting, persistence hooks, styling, accessibility, and a practical application recipe.

Table of Contents

1. Introduction 2

1.1 Key features 2

1.2 Maven dependency 2

2. Getting started 2

3. Anatomy 3

4. Control API 4

4.1 DockingContext 4

4.2 DockingPane 4

5. Tab and split interactions 5

6. Floating stages 6

7. Persistence 6

7.1 Multi-child split snapshots 7

7.2 Divider updates 7

7.3 Capture and restore 8

8. Styling 8

9. Accessibility and localization 9

10. Basic app recipe 10

11. See also 10

1. Introduction

DockingPane is a JavaFX `Control` that renders dockables as custom title tabs (not a JavaFX `TabPane`). A `DockingContext` owns one or more panes, resolves dockables by ID, coordinates cross-pane movement, and creates floating windows through an application-supplied `DockingStageFactory`.

1.1 Key features

- Custom tab strip with title, optional graphic, toolbar, help, menu, and close controls.
- Multi-child horizontal/vertical splits with ordered divider positions in [0, 1].
- Drag-and-drop within one pane and across panes in the same context.
- Shift-drag to move an entire tab group; drop onto an empty pane's placeholder to populate it.
- Four outer-edge indicators for docking beside the entire pane layout.
- Detaching tabs into app-created floating stages.
- Context-wide snapshot and restore via `getLayout()` and `restoreLayout()`.
- Close policy hook via `setCanCloseDockable(...)`.

1.2 Maven dependency

```
<dependency>
    <groupId>com.dlsc.gemsfx</groupId>
    <artifactId>gemsfx</artifactId>
    <version>4.7.0</version>
</dependency>
```

DockingPane lives in module `com.dlsc.gemsfx`, package `com.dlsc.gemsfx.docking`.

2. Getting started

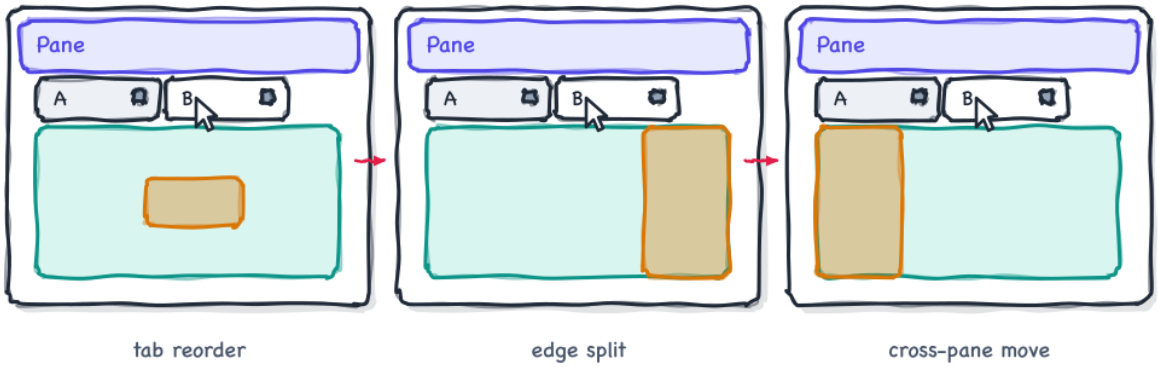
A docking setup starts with two factories: one that creates dockables by ID and one that returns a stage hosting each floating pane.

```
DockableFactory dockableFactory = id -> {
    if ("editor".equals(id)) {
        return Optional.of(new Dockable("editor", "Editor", new TextArea()));
    }
    if ("inspector".equals(id)) {
        return Optional.of(new Dockable("inspector", "Inspector", new ListView<>()));
    }
    return Optional.empty();
};

DockingStageFactory stageFactory = pane -> {
    Stage stage = new Stage();
    stage.setTitle("Floating Dock");
    stage.setScene(new Scene(new BorderPane(pane), 420, 280));
    stage.show();
    return stage;
};

DockingContext context = new DockingContext(dockableFactory, stageFactory);
DockingPane pane = context.createPane();
context.openDockable(pane, "editor", DockingPosition.CENTER);
context.openDockable(pane, "inspector", DockingPosition.RIGHT);
```

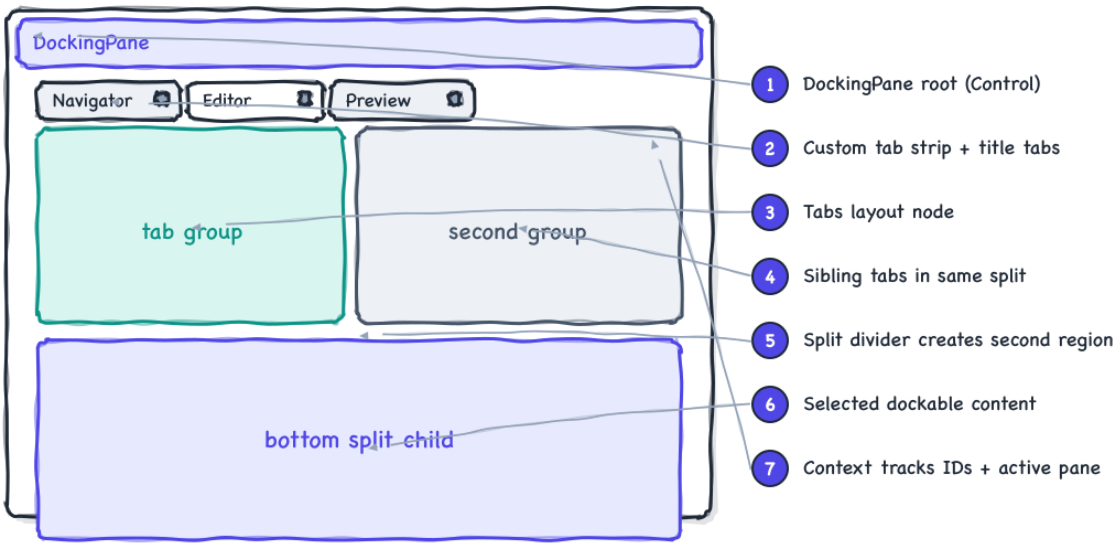
Minimal setup with an ID-based DockableFactory and a shared DockingStageFactory.



Dragging a title reorders tabs, creates splits, or moves a dockable into another pane.

> **Note.** DockingContext methods mutate model state and are expected to run on the JavaFX application thread.

3. Anatomy



Main structural parts of DockingPane.

Part	Type	Description
DockingPane	Control	Root control with style class `docking-pane` and stylesheet `docking-pane.css`.
DockingContext	Model coordinator	Tracks open dockables, pane membership, close policy, floating stages, and snapshot/restore.
Tab group	Layout node	A set of dockable IDs plus one selected ID.
Split node	Layout node	An orientation, at least two ordered children, and one fewer normalized divider positions than children.

Part	Type	Description
Empty-pane placeholder	Node	Application-replaceable content shown in an empty pane; accepts dockable drops from the same context.
Floating pane	DockingPane	Created by the context when a dockable is detached to a stage.

The public layout tree is represented by immutable `DockingLayoutNode` variants: `Empty`, `Tabs`, and `Split`.

4. Control API

4.1 DockingContext

Property	Type	Default	Description
createPane()	DockingPane	new pane	Creates a pane attached to the context.
openDockable(pane, id, position)	Optional<Dockable>	n/a	Opens the requested dockable ID or activates the existing one if already open.
floatDockable(id, x, y)	DockingPane	n/a	Moves one open dockable into a new floating pane and positions its stage.
canMoveTabGroup(...)	boolean	false if invalid	Checks whether a tab group can move into another group, beside a pane, or into an empty pane.
moveTabGroup(...)	boolean	false if invalid	Moves the entire source group while preserving tab order and selection.
getLayout()	DockingLayout	snapshot	Captures all panes and floating stage bounds.
restoreLayout(layout)	DockingRestoreResult	n/a	Rebuilds the context from a snapshot and reports unresolved IDs.
canCloseDockable policy	Predicate<Dockable>	always true	Veto hook used by close buttons, keyboard close, and floating stage close.

4.2 DockingPane

Property	Type	Default	Description
activeDockable	ReadOnlyObjectProperty<Dockable>	null	Currently active dockable of this pane.
layoutRoot	ReadOnlyObjectProperty<DockingLayoutNode>	Empty	Immutable snapshot of this pane tree, updated on structure, selection, and divider changes.
placeholder	ObjectProperty<Node>	DockingPanePlaceholder	Content shown in an empty pane. Replace it with a custom node, or set null for a blank pane.
selectDockable(id)	void	n/a	Activates a dockable by ID.
updateDividerPositions(path, positions)	void	n/a	Atomically replaces a split's entire divider vector and publishes one layout snapshot.
moveDockable(...)	boolean	false if invalid	Moves a dockable by ID relative to a target dockable.

Property	Type	Default	Description
canMoveDockableToPaneEdge(id, edge)	boolean	false if invalid	Checks whether a dockable can move beside the entire pane layout.
moveDockableToPaneEdge(id, edge)	boolean	false if invalid	Moves a dockable to an outer LEFT, RIGHT, TOP, or BOTTOM edge, spanning all existing groups.

5. Tab and split interactions

Dragging a title inside the pane resolves a target from cursor position and the currently hovered title. Center drops merge into tabs, while local edge drops split the targeted group. Four additional indicators appear near the outer edges during a valid docking drag. Dropping on one of these indicators docks beside the entire pane layout, not just the group under the pointer.

Gesture	Result
Drop on center zone	Dockable joins target tab group and becomes selected.
Drop on left/right zone	Horizontal split is created around the target group.
Drop on top/bottom zone	Vertical split is created around the target group.
Drop on outer top/bottom indicator	New group spans the full pane width above/below all remaining groups.
Drop on outer left/right indicator	New group spans the full pane height beside all remaining groups.
Drop on another pane (same context)	Cross-pane move; source pane may become empty.
Drop on an empty pane's placeholder	Moves the dockable into the empty pane and selects it. Placeholder child nodes also accept drops.
Shift-drag a title	Moves its entire tab group, preserving tab order and the group's selected tab.
ESC during drag	Cancels external floating; no detachment happens.

For example, with two groups side by side, drag a tab onto the bottom outer-edge indicator to place it below both groups. The preview spans the entire pane width. The source tab is removed first and empty branches collapse. The new group is added to a matching-orientation split, or a new split is created when orientations differ. The same operation works when dragging from a floating pane in the same context. Indicators disappear after dropping, leaving the pane, or canceling.

```
pane.moveDockableToPaneEdge("console", DockingPosition.BOTTOM);
```

Place the console below the pane's entire remaining layout.

Consecutive splits with the same orientation are flattened into one multi-child split. Moving a divider therefore resizes the two adjacent items rather than a nested same-axis subtree, subject to the items' minimum and maximum sizes. Opposite-orientation splits remain nested and resize as a unit. Inserting a group beside a dockable subdivides only that group's existing interval.

Minimum and preferred sizes honor the selected content's bias throughout the split hierarchy. Horizontally biased content is measured at each group's allocated width; vertically biased content is measured at its allocated height. This also applies to an empty pane's placeholder. Switching tabs updates the propagated bias to match the newly selected content.

A normal drag onto an empty pane moves only the dragged dockable; Shift-drag moves its whole tab group. Both show a full-pane preview and work over the placeholder itself or any of its child nodes. The source and destination must belong to the same DockingContext.

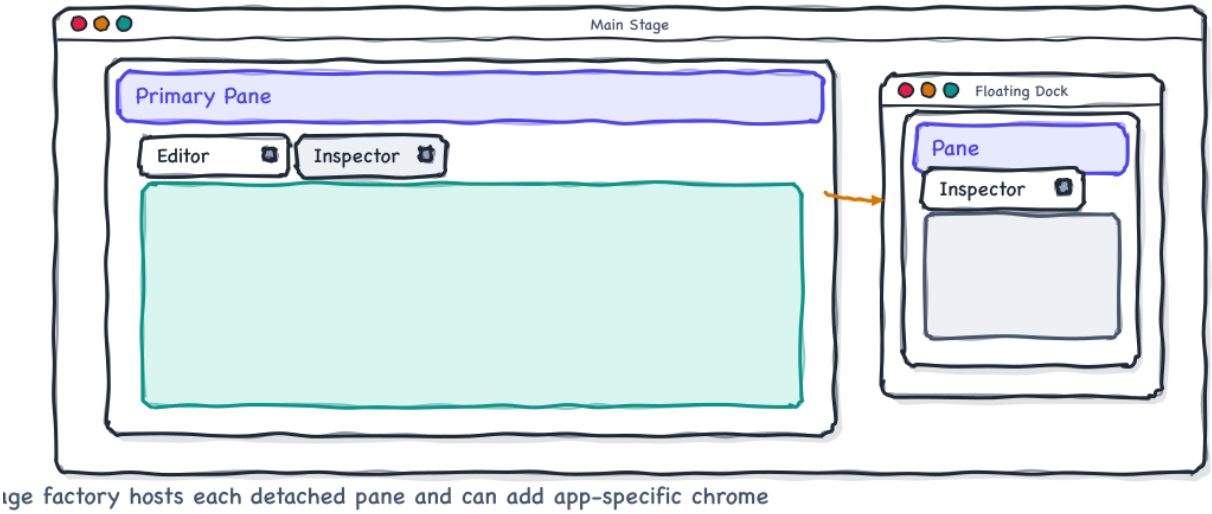
```
pane.setPlaceholder(new DockingPanePlaceholder("Drop dockables here"));
```

Customize the default illustrated placeholder's text, or supply any other Node.

Keyboard interactions of the title controls: LEFT/RIGHT switches selected tab, ENTER or SPACE selects the focused title, DELETE or shortcut+W requests close.

Focusing a dockable's content or title makes it active. If another dockable is activated while focus is in a dockable, focus moves into the newly active content or, if it has no focusable node, to its title. Focus outside the pane is unchanged.

6. Floating stages

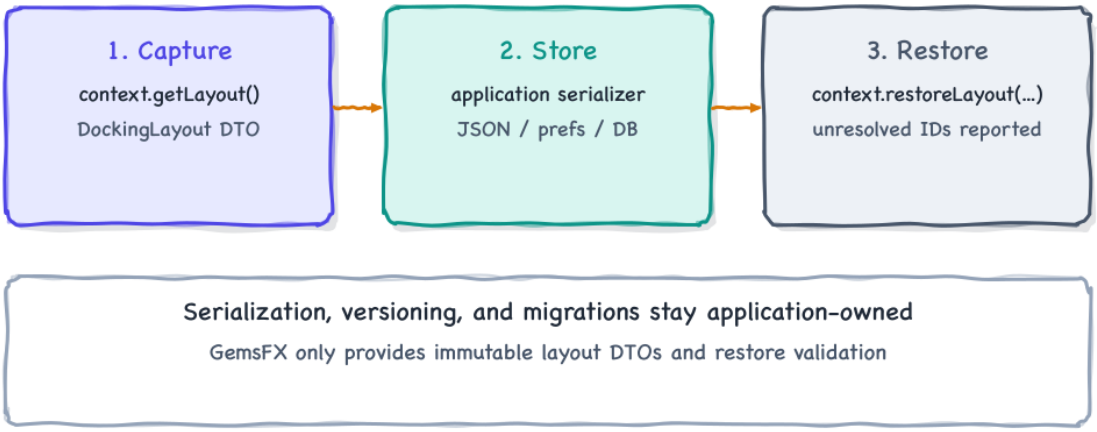


A detached dockable is hosted by a stage produced by the shared stage factory.

When a drag ends outside all context panes, or when the app calls `floatDockable(...)`, the context creates a fresh pane and asks the stage factory to host it. The factory must return a stage that already contains the supplied pane.

- Floating stages are tracked per host stage and close automatically when the host closes.
- If the last dockable leaves a floating pane, its stage is closed automatically.
- Close-veto policy is respected before a floating stage closes.

7. Persistence



Applications own persistence: capture, serialize externally, deserialize, then restore.

DockingContext intentionally keeps serialization/versioning outside the library. Use `getLayout()` to capture a DTO graph, then serialize it in your own format.

7.1 Multi-child split snapshots

`DockingLayoutNode.Split` exposes `getOrientation()`, `getChildren()`, and `getDividerPositions()`. The child and divider lists are immutable defensive copies. For N children there must be exactly N - 1 divider positions, in nondecreasing order within [0, 1]. NaN and infinite values are rejected; equal positions and the endpoints 0 and 1 are valid and represent collapsed intervals.

```
DockingLayoutNode.Split root = new DockingLayoutNode.Split(
        Orientation.HORIZONTAL,
        List.of(
                new DockingLayoutNode.Tabs(List.of("editor"), "editor"),
                new DockingLayoutNode.Tabs(List.of("console"), "console"),
                new DockingLayoutNode.Tabs(List.of("inspector"), "inspector")),
        List.of(0.35, 0.8));
```

Three groups occupying 35%, 45%, and 20% of a horizontal split.

Note. This is a breaking snapshot/API change. Split nodes use children and dividerPositions, not first, second, or dividerPosition. There are no legacy constructors, accessors, or automatic format adapters; update application-owned serializers accordingly.

7.2 Divider updates

`updateDividerPositions(path, positions)` validates the entire divider vector before changing the model and publishes one new snapshot per accepted update. Invalid vectors leave the previous layout unchanged. The path is a list of ordered child indices: an empty list addresses the root; `List.of(2)` addresses a split in the root's third child. Path indices are not limited to 0 and 1.

```
pane.updateDividerPositions(List.of(), List.of(0.35, 0.8));
```

Update both dividers of an existing three-child root split atomically.

7.3 Capture and restore

```
DockingLayout layout = context.getLayout();
String json = mySerializer.write(layout); // application-owned

DockingLayout restored = mySerializer.read(json);
DockingRestoreResult result = context.restoreLayout(restored);
if (!result.getUnresolvedDockableIds().isEmpty()) {
    log.warn("Missing dockables: {}", result.getUnresolvedDockableIds());
}
```

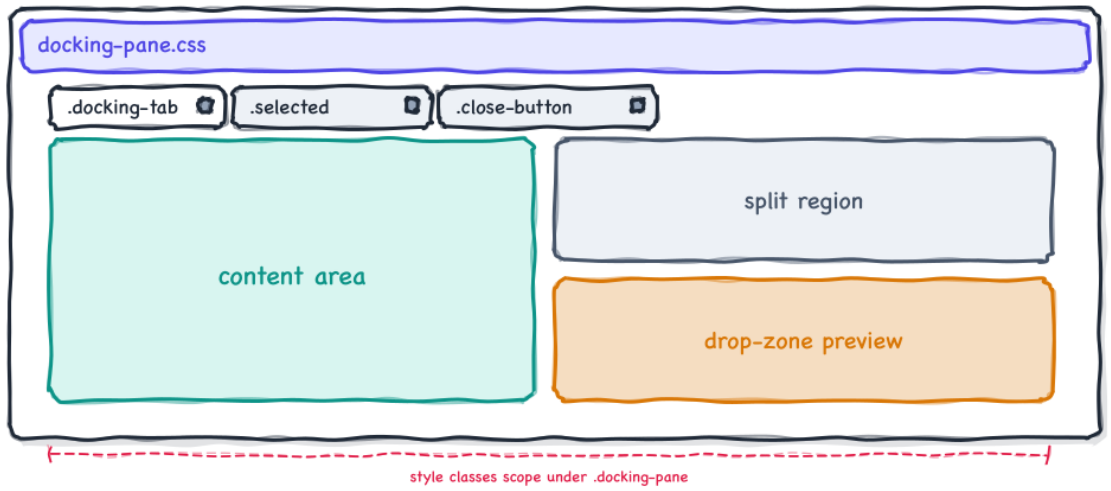
Snapshot and restore without introducing a library-side serializer.

> **Note.** Restore validates the layout shape and divider range, and requires exactly one primary pane (the pane layout with null stage bounds).

Restoration accepts same-orientation nesting and canonicalizes it into a flat split, transforming nested divider positions into their parent intervals. Opposite-orientation children remain nested. Captured layouts use the same canonical structure as the renderer, with one JavaFX SplitPane per split node.

Unresolved dockables are reported and removed from the restored layout. Empty groups are removed and one-child splits collapse. A removed group's interval goes to its preceding sibling, or to the following sibling when the first group is removed; other boundaries stay unchanged. The same space-allocation rule applies when dockables are closed or moved away.

8. Styling



Main style classes for tabs, split panes, and drop previews.

Selector	Meaning
`.docking-pane`	Control root background and border.
`.docking-tab-strip`	Tab strip container and spacing.
`.docking-tab-pane:active`	Tab group containing the pane's active dockable.
`.docking-tab`	Custom title tab shape and hover/selected visuals.

Selector	Meaning
`.docking-tab:selected`	The selected tab in each tab group.
`.docking-tab-actions`	Shared fading container for the toolbar, help, menu, and close controls.
`.docking-tab-help-button`	Help action supplied by the dockable's onHelp callback.
`.docking-tab-menu-button`	Menu populated from the dockable's menuItems list.
`.docking-tab-close-button`	Close affordance and icon.
`.docking-split`	SplitPane wrappers created for split layout nodes.
`.docking-drop-leaf-preview`	Outline of target leaf during drag.
`.docking-drop-zone-preview`	Filled drop zone preview.
`.docking-pane-edge-target`	Pane-wide drop indicator; :active highlights the hovered target.
`.pane-edge-left/right/top/bottom`	Direction-specific style classes of the outer indicators.

Toolbar, help, menu, and close controls share one hover fade. The actions container becomes unmanaged when its opacity reaches zero, releasing width for the title without changing tab height. Configure optional actions through `Dockable.setToolbar(...)`, `setOnHelp(...)`, and `getMenuItems()`.

The default tab height is fixed by CSS at 2.5em, so it scales with the font size rather than toolbar content. Child controls are vertically centered, use their preferred widths, and are constrained to the available height. Override the preferred height to change all tabs consistently.

```
.docking-pane .docking-tab {
    -fx-pref-height: 2.75em;
    -fx-min-height: -fx-pref-height;
    -fx-max-height: -fx-pref-height;
}
```

Keep minimum and maximum heights tied to the font-scaled preferred height.

```
.docking-pane .docking-tab:selected .docking-tab-title {
    -fx-font-weight: bold;
}
.docking-pane .docking-drop-zone-preview {
    -fx-background-color: -fx-accent;
    -fx-opacity: 0.35;
}
```

Excerpt from `docking-pane.css`.

9. Accessibility and localization

DockingPane sets `AccessibleRole.NODE` and a localized role description from resource key `accessible.role-description`.

- Each title tab uses `AccessibleRole.TAB_ITEM` and binds accessible text to the dockable title.
- Close buttons use localized tooltip text (`tooltip.close`).
- Close buttons format accessible text through `accessible.text.close` (for example: *Close Inspector*).

Resource key	Default value
`accessible.role-description`	Docking pane
`accessible.text.close`	Close {0}
`tooltip.close`	Close

10. Basic app recipe

1. Define a stable ID catalog for all dockables your app can create.
2. Implement `DockableFactory` that maps IDs to new Dockable instances.
3. Implement one shared `DockingStageFactory` that wraps supplied panes in app-specific stage chrome.
4. Create one DockingContext, one primary DockingPane, and open the initial dockables.
5. Persist `DockingLayout` snapshots using your own serializer and restore on startup.

```
DockingContext context = new DockingContext(dockableFactory, stageFactory);
DockingPane rootPane = context.createPane();

context.openDockable(rootPane, "editor", DockingPosition.CENTER);
context.openDockable(rootPane, "navigator", DockingPosition.LEFT);
context.openDockable(rootPane, "console", DockingPosition.BOTTOM);

// later
DockingLayout snapshot = context.getLayout();
context.restoreLayout(snapshot);
```

Minimal recipe aligned with the DockingPane demo application.

11. See also

- Demo application: `com.dlsc.gemsfx.demo.DockingPaneApp` (run: `mvn javafx:run -f gemsfx-demo/pom.xml -Dmain.class=com.dlsc.gemsfx.demo.DockingPaneApp`).
- Related APIs: `DockingContext`, `DockingLayout`, `DockingLayoutNode`, `DockingRestoreResult`.
- API documentation: https://dlsc-software-consulting-gmbh.github.io/GemsFX/api/